Unit-04

Functions in Python

A **function** is a block of code that performs a specific task. Functions help to make code reusable, modular, more readable, easy, scalable and abstraction. You can define a function in Python using the `def` keyword.

Syntax:`

```
def function_name():#definition
    #body
function_name() #call
```

Example 1:

```
def hello():
    print("Hello, welcome to Python!")
# Calling the function
hello()
```

Output:

```
Hello, welcome to Python!
```

Example 2:

```
def add(a, b):
    return a + b
print(add(5, 3))  # Output: 8
b=add(10, 20)
print(b)          # Output: 30
```

Example 3:

```
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        return "Cannot divide by zero"
    return a / b

def calculator():
    print("Select operation:")
```

```
    print("1. Add")
    print("2. Subtract")
    print("3. Multiply")
    print("4. Divide")
    choice = input("Enter choice (1/2/3/4): ")
    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))

    if choice == '1':
        print(f"Result: {add(num1, num2)}")
    elif choice == '2':
        print(f"Result: {subtract(num1, num2)}")
    elif choice == '3':
        print(f"Result: {multiply(num1, num2)}")
    elif choice == '4':
        print(f"Result: {divide(num1, num2)}")
    else:
        print("Invalid input")

# Running the calculator
calculator()
```

Output:

```
Select operation:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice (1/2/3/4): 1
Enter first number: 10
Enter second number: 5
Result: 15.0
```

Why use Function

1. Reusability

- Functions let you write code once and reuse it multiple times, avoiding duplication.
- This reduces errors and makes the code concise.

Example:

```
def add(a, b):
    return a + b
# Reusing the same function
print(add(5, 3))  # Output: 8
print(add(10, 20))  # Output: 30
```

---

2. Modularity

- Functions break a complex program into smaller programs.
- Each function can handle a specific task, making the program easier to read and debug.

Example:

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    return a / b if b != 0 else "Division by zero is not allowed"
# Modularized calculator operations
print(add(5, 3))  # Output: 8
print(divide(10, 2))  # Output: 5.0
```

---

3. Abstraction

- Functions hide the complexity of a task, allowing users to focus on what the function does rather than how it works like print( ) and input( ) etc.

Example:

```
def calculate(operation, a, b):
    if operation == 'add':
        return add(a, b)
    elif operation == 'subtract':
        return subtract(a, b)
    elif operation == 'multiply':
        return multiply(a, b)
    elif operation == 'divide':
        return divide(a, b)
    else:
        return "Invalid operation"

# Abstracting the implementation details
print(calculate('add', 10, 5))  # Output: 15
print(calculate('divide', 10, 0))  # Output: Division by zero is not allowed
```

---

4. Scalability

- As programs grow, functions make it easier to add new features or extend functionality without rewriting everything.

Example: Adding Advanced Calculator Functions

```
import math
def square_root(a):
    return math.sqrt(a)
def power(a, b):
    return math.pow(a, b)
# Scaling up
print(square_root(16))  # Output: 4.0
print(power(2, 3))  # Output: 8.0
```

Practice Problems

1. **Addition Function** Write a function `add(a, b)` that takes two numbers as input and returns their sum. **Hint**: Use the `return` statement to output the result.
2. **Check Even or Odd** Create a function `is_even(n)` that checks if a given number is even. **Hint**: Use the modulus operator (`%`) to check divisibility by 2.
3. **Maximum of Two Numbers** Write a function `max_of_two(a, b)` that returns the larger of two numbers. **Hint**: Use an `if-else` statement to compare the numbers.
4. **Square of a Number** Create a function `square(n)` that takes a number and returns its square. **Hint**: Use `n * n` or `n ** 2` for the calculation.
5. **Convert Celsius to Fahrenheit** Write a function `celsius_to_fahrenheit(celsius)` to convert temperature from Celsius to Fahrenheit. **Hint**: Use the formula: F=C*9/5+32.
6. **Find the Factorial of a Number** Write a function `factorial(n)` that calculates the factorial of a number iteratively. **Hint**: Use a `for` loop to multiply numbers from 1 to `n`.
7. **Count Vowels in a String** Create a function `count_vowels(s)` that returns the number of vowels in a string. **Hint**: Iterate through the string and check if each character is in `'aeiou'`.
8. **Check Prime Number** Write a function `is_prime(n)` to check if a number is prime. **Hint**: Check if the number is divisible by any number between 2 and n.
9. **Reverse a String** Create a function `reverse_string(s)` to return the reverse of a string. **Hint**: Use slicing (`s[::-1]`) or a `for` loop to reverse the string.
10. **Calculate Simple Interest** Create a function `simple_interest(principal, rate, time)` to calculate simple interest. **Hint**: Use the formula: SI=P×R×T/100.

Function Arguments and parameters

- **Parameters** are variables in the function definition that receive the values in parenthesis.
- **Arguments** are the actual values passed to the function when calling it.

Example:

```
def greet(name):  # `name` is a parameter
    print(f"Hello, {name}!")
greet("Alice")  # "Alice" is an argument
Output:  Hello, Alice!
```

Types of Arguments and parameters

(a) Positional Arguments

- Passed arguments to a function in the **order** they are defined.
- The number of arguments must **match** the number of parameters.

```
def add(a, b):
    return a + b
result = add(3, 5)
print(result)  # Output: 8
add(3)  # TypeError: missing 1 required positional argument
```

(b) Default Arguments

- Parameters can have **default values**.
- If an argument is not provided, the default value is used.

```
def greet(name="Guest"):
    print(f"Hello, {name}!")
greet()          # Output: Hello, Guest!
greet("Bob")     # Output: Hello, Bob!
```

(c) Keyword Arguments

- Arguments can be passed (Function call) using **key-value pairs**, allowing you to specify which parameter each value is for.

```
def describe(name, age, job):
    print(f"Name: {name}, Age: {age}, Job: {job}")
describe(name="Alice", job="Engineer", age=30)
```

Output:

```
Name: Alice, Age: 30, Job: Engineer
```

- Keyword arguments make the function call more **readable**.
- The **order** does not matter when using keyword arguments.

(d) Variable-Length Arguments

1. `*args` (Non-Keyword Variable-Length Arguments)

- Collects **extra positional arguments** as a **tuple**.
- Used when you don't know the number of arguments .

```
def add_numbers(*args):
    print(args)
    return sum(args)

print(add_numbers(1, 2, 3, 4, 5))
```

Output:

```
(1, 2, 3, 4, 5)
15
```

- `*args` collects additional positional arguments as a **tuple**.
- You can name it anything, but `*args` is the convention.

Example 2:
Taking input from user

```
def add_numbers(*args):
   return sum(args)  # Adds all numbers passed as arguments
# Taking input from user and converting it to integers
numbers = map(int, input("Enter numbers separated by space: ").split())
# Passing user input as positional arguments using *
result = add_numbers(*numbers)
print("Sum:", result)
```

Output:

```
Enter numbers separated by space: 5 10 15
Sum: 30
```

2. `**kwargs` (Keyword Variable-Length Arguments)

- Collects **extra keyword arguments** as a **dictionary**.
- Useful when you want to handle a variable number of named arguments.
- `kwargs.items()` allows you to iterate over key-value pairs.

```
def show_info(**kwargs):
    print(kwargs)
show_info(name="Alice", age=25, job="Engineer")
```

Output:

```
{'name': 'Alice', 'age': 25, 'job': 'Engineer'}
```

3. `*args` and `**kwargs` Together

Example

```
def show_details(*args, **kwargs):
```

```
    print("Non-Keyword arguments:", args)
    print("Keyword arguments:", kwargs)
show_details("Alice", "Bob", age=30, city="New York")
```

Output:

```
Non-Keyword arguments: ('Alice', 'Bob')
Keyword arguments: {'age': 30, 'city': 'New York'}
```

Variable Scope

Variable scope refers to **where a variable can be accessed** within a program. In Python, there are **four types of variable scopes**:

1. **Local Scope**
2. **Enclosing Scope (Nonlocal)**
3. **Global Scope**
4. **Built-in Scope**

---

1. Local Scope

Variables declared **inside a function** are considered **local** and can be accessed only within that function.

```
def my_function():
    x = 10  # Local variable
    print(x)  # Accessible inside the function

my_function()
# print(x)  # This will raise an error (NameError: name 'x' is not defined)
x is a local variable and exists only inside my_function().
```

2. Enclosing Scope (Nonlocal)

- When a function is **nested inside another function**, the inner function has access to the **variables of the outer function**.

```
def outer():
    x = 5  # Enclosing variable
    def inner():
        nonlocal x  # Refers to 'x' in outer function
        x += 1
        print("Inner x:", x)
    inner()
    print("Outer x:", x)
outer()
```

Output:

```
Inner x: 6
Outer x: 6
```

---

3. Global Scope

- Variables declared **outside any function or class** are **global variables**.
- They can be accessed anywhere in the code.

```
x = 100  # Global variable
def my_function():
    print("Inside function:", x)  # Can access global variable
my_function()
print("Outside function:", x)
```

Output:

```
Inside function: 100
Outside function: 100
```

---

4. Built-in Scope

- Python has a set of **built-in functions and keywords** that are available everywhere.
- These are stored in Python's **built-in scope**.

```
print(len([1, 2, 3]))  # 'len' is a built-in function
print(abs(-5))  # 'abs' is a built-in function
```

Output:

```
3
5
```

Python follows the **LEGB rule** to determine variable scope:

1. **L → Local** (Inside the function)
2. **E → Enclosing** (Inside an enclosing function)
3. **G → Global** (Defined at the top level)
4. **B → Built-in** (Predefined names in Python)

```
x = "Global"  # Global variable
def outer():
    x = "Enclosing"  # Enclosing variable
    def inner():
        x = "Local"  # Local variable
        print(x)  # Local scope is checked first
    inner()
```

```
outer()
```

Output:

```
Local
```

---

9. Recursive Functions

Recursion is a programming technique where a function calls itself to solve a problem. Recursive functions are useful for problems that can be broken down into smaller, similar sub-problems.

Key Components of Recursion

1. **Base Case**: This is the condition that stops the recursion. Without a base case, the recursion would continue indefinitely and lead to a stack overflow error.
2. **Recursive Case**: This is the part of the function where it calls itself and working toward the base case.

Advantages of Recursion

1. **Elegance**: Recursive solutions can be more concise and easier to read for problems like tree traversal, factorials, and Fibonacci sequences.
2. **Divide-and-Conquer**: Recursion is a natural fit for problems that can be divided into similar sub-problems, such as sorting algorithms (e.g., QuickSort, MergeSort).

Disadvantages of Recursion

1. **Performance**: Recursive calls consume stack memory, which can lead to stack overflow errors for deep recursions.
2. **Overhead**: Recursive functions often have more function call overhead compared to iterative solutions.
3. **Complexity**: Debugging recursive functions can be challenging, especially for problems with multiple base cases.

Syntax of a Recursive Function

```
def function_name(parameters):
    if base_condition:  # Base case
        return some_value optional
    else:
        # Recursive case
        return recursive_function(modified_parameters)
```

---

1. Print "Hello" N times

```
def print_hello(n):
    if n <= 0:  # Base case
        return
    print("Hello")
    print_hello(n - 1)  # Recursive call
print_hello(3)
```

Output:

```
Hello
Hello
Hello
```

---

2. Simple Countdown

```
def countdown(n):
    if n < 0:  # Base case
        return
    print(n)
    countdown(n - 1)  # Recursive call
countdown(3)
```

Output:

```
3
2
1
0
```

---

3. Simple Count Up

```
def countup(n, target):
    if n > target:  # Base case
        return
    print(n)
    countup(n + 1, target)  # Recursive call
countup(1, 3)
```

Output:

```
1
2
3
```

---

4. Add Numbers from 1 to N

```
def add_numbers(n):
    if n == 0:  # Base case
        return 0
    return n + add_numbers(n - 1)  # Recursive call
print(add_numbers(3))
# Output: 6
```

---

5. Printing Even Numbers Recursively

```
def print_even(n):
    if n < 0:  # Base case
        return
    if n % 2 == 0:
        print(n)
    print_even(n - 1)  # Recursive call
print_even(10)
```

Output:

```
10
8
6
4
2
0
```

Example 1: Factorial Using Recursion

The factorial of a number n is calculated as $n!=n\times(n-1)\times(n-2)\times...\times1$.

```
def factorial(n):
   if n == 1:  # Base case
      return 1
   else:
      return n * factorial(n - 1)  # Recursive case
print(factorial(5))
 # Output: 120
```

Example 2: Fibonacci Sequence Using Recursion

```
def fibonacci(n):
   if n <= 0:
      return "Invalid input, enter a positive integer."
   elif n == 1:
      return 0
   elif n == 2:
```

```
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)
# Get user input
num= int(input("Enter the number of terms: "))
print("Fibonacci Series:")
for i in range(1, num+ 1):
    print(fibonacci(i), end=" ")
```

Output:

0 1 1 2 3 5 8 13 21 34

Practice Problems

1. **Factorial of a Number**
   - Hint: Multiply the number by the factorial of the number minus one, stopping when the number is 1.
2. **Fibonacci Sequence**
   - Hint: The Fibonacci number is the sum of the two preceding numbers. Use base cases for n=0 and n=1.
3. **Sum of Digits in a Number**
   - Hint: Use `n % 10` to extract the last digit and `n // 10` to reduce the number.
4. **Power of a Number**
   - Hint: Multiply the base by the result of the function with one less exponent, stopping when the exponent is 0.
5. **Sum of the First `n` Natural Numbers**
   - Hint: Add `n` to the result of the function for n−1, stopping when n=0.
6. **Reverse a String**
   - Hint: Concatenate the last character of the string with the reverse of the substring excluding the last character.
7. **Tower of Hanoi**
   - Hint: Move `n-1` disks from source to auxiliary, move the nth disk to the destination, then move `n-1` disks from auxiliary to destination.

Types of Functions in Python

1. **Built-in Functions**: Functions provided by Python, such as `print()`, `type()`, `input()` etc.
2. **User-defined Functions**: Functions that you create to perform specific tasks.
3. **Anonymous Functions**: Also called **lambda functions**, which are functions without a name.

3. Anonymous Functions

An **anonymous function** is a function defined without a name. These functions are created using the `lambda` keyword, often referred to as **lambda functions**. They are primarily used when you need a small, short-lived function(Do not use def keyword).

Syntax of a Lambda Function

```
lambda arguments: expression
```

- **`lambda`**: Keyword used to define the function.
- **`arguments`**: A comma-separated list of parameters the function takes.
- **`expression`**: A single expression that the function evaluates and returns. (No multi-line logic is allowed.)

Key Characteristics of Lambda Functions

1. **Anonymous**: They don't have a name unless you assign one.
2. **Inline**: Typically written in a single line for brevity.
3. **Single Expression**: Limited to one expression, unlike normal functions defined using `def`.

Examples of Lambda Functions

1. Basic Lambda Function

```
# Lambda function to add two numbers
add = lambda x, y: x + y
print(add(5, 3))  # Output: 8
```

2. Lambda Function Without Assignment

You can directly use a lambda function without assigning it to a variable.

```
print((lambda x, y: x * y)(4, 5))  # Output: 20
```

Practice Problems

1. `Squaring of list numbers`

```
nums = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, nums))
print(squared)  # Output: [1, 4, 9, 16]
```

2. `if-else`

```
max_value = lambda x, y: x if x > y else y
print(max_value(10, 20))  # Output: 20
```

3. `filter()`

```
nums = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)  # Output: [2, 4, 6]
```

4. `sorted()`

```
words = ["banana", "apple", "cherry"]
sorted_words = sorted(words, key=lambda x: len(x))
print(sorted_words)  # Output: ['apple', 'banana', 'cherry']
```

5. `return multiple values`

```
func = lambda x: (x**2, x**3, x**4)
print(func(2))  # Output: (4, 8, 16)
```

Comparison with Regular Functions

Aspect	Lambda Function	Regular Function (`def`)
Naming	Anonymous (no name unless assigned).	Requires a name.
Lines	Single-line expression only.	Can include multiple lines and statements.
Reusability	Limited; not ideal for complex logic.	Ideal for reusable and complex logic.
Readability	Compact but can be harder to read.	More explicit and easier to understand.

Class

Python is an **object-oriented programming (OOP)** language. A **class** is a blueprint (prototype) for creating objects or class is description of ojbect. It defines **attributes (variables)** and **methods (functions)** that objects of the class.

Real World Example: A **Blueprint** is a design plan of home . It shows what the house will look like Number of rooms, Color of the house, Type of windows and doors. But it's just a **plan**; you can't live in it! is called class, **object** is the **actual home** made using the blueprint.

- Variables & Function Names: `snake_case`
- Class Names: `PascalCase` (`TitleCase`)

- **Constants:** `SCREAMING_SNAKE_CASE`
- **Magic Methods (Dunder Methods):** `__init__`, `__str__`

Difference Between a Function and a Method

Feature	Function	Method
Defined Inside a Class?	❌ No	✅ Yes
Requires `self`	❌ No	✅ Yes (for instance & class methods)
Called on an Object?	❌ No	✅ Yes
Accesses Instance/Class Variables?	❌ No	✅ Yes
Example:1	`def greet():`	`def greet(self):`
Example:2	A=[1,2,3] len(A)	`A.append()`

Syntax of a Class:

```
#creating class
class ClassName:
    # Class body
#creating object
object_name = ClassName()
#accessing class members
object_name.attribute_name
object_name.method_name()
```

Example:

```
class Car:
    car_count = 0  # class variable
    @classmethod
    def start(cls):  # class method
        print(f"The car has stared {cls.car_count}.")
        print("The car has started.")
    @classmethod
    def stop(cls):  # class method
        print("The car has stopped.")
Car.start()  # Output: The car has started.
Car.stop()   # Output: The car has stopped.
car_obj=Car() #creating object
car_obj.start()
car_obj.stop()
```

2. Constructor (`__init__` Method)

- The **constructor** is a special method in Python called `__init__()`.
- It is **automatically called** when you **create an object** from a class.
- It is used to **initialize (set up)** the object with **initial values**.

Why Use a Constructor?

- **Initialization:** Automatically sets the values when the object is created.
- **Configuration:** we can set part of application configuration.

Think of it like **setting up a new phone**:
When you buy a new phone, you set up your language, Wi-Fi, and accounts.
Similarly, when you create an object, the `__init__` method sets up its initial data.

- It's always named `__init__()` (with double underscores).
- It is defined inside a **class**.
- The first parameter is always `self`, which refers to the **current object**.

Example:

```
class ATM:
    def __init__(self, balance=0):
        self.balance = balance
        self.pin = "1234"  # Default PIN (for demo purposes)

    def authenticate(self):
        entered_pin = input("Enter your 4-digit PIN: ")
        if entered_pin == self.pin:
            print("Authentication successful.\n")
            return True
        else:
            print("Incorrect PIN.\n")
            return False

    def check_balance(self):
        print(f"Your current balance is: ₹{self.balance}\n")

    def deposit(self):
        amount = float(input("Enter amount to deposit: ₹"))
        if amount > 0:
            self.balance += amount
            print(f"₹{amount} deposited successfully.\n")
```

```
        else:
            print("Invalid deposit amount.\n")

    def withdraw(self):
        amount = float(input("Enter amount to withdraw: ₹"))
        if 0 < amount <= self.balance:
            self.balance -= amount
            print(f"₹{amount} withdrawn successfully.\n")
        else:
            print("Insufficient balance or invalid amount.\n")

    def create_new_pin(self):
        current_pin = input("Enter your current PIN: ")
        if current_pin == self.pin:
            new_pin = input("Enter new 4-digit PIN: ")
            confirm_pin = input("Confirm new PIN: ")
            if new_pin == confirm_pin and len(new_pin) == 4 and
new_pin.isdigit():
                self.pin = new_pin
                print("PIN changed successfully.\n")
            else:
                print("PINs did not match or invalid format. Try again.\n")
        else:
            print("Incorrect current PIN.\n")

    def run(self):
        if not self.authenticate():
            return
        while True:
            print("==== ATM Menu ====")
            print("1. Check Balance")
            print("2. Deposit")
            print("3. Withdraw")
            print("4. Change PIN")
            print("5. Exit")
            choice = input("Enter your choice (1-5): ")

            if choice == "1":
                self.check_balance()
            elif choice == "2":
                self.deposit()
            elif choice == "3":
                self.withdraw()
            elif choice == "4":
                self.create_new_pin()
```

```
            elif choice == "5":
                print("Thank you for using the ATM. Goodbye!\n")
                break
            else:
                print("Invalid choice. Please try again.\n")


# Run the ATM
atm = ATM(balance=1000)  # Starting with ₹1000
atm.run()
```

Types of attributes and methods

1. Class

Class-level attributes and methods are associated with the **class itself**, not with individual instances

1. Class Attributes (Class Variables)

Class attributes, more accurately called **class variables**, are variables that are shared by **all instances** of the class. These variables are defined **outside of any instance method**, usually directly under the class definition.

2. Class Methods

Class methods are methods that are bound to the class and not the instance. They are defined using the `@classmethod` decorator and take `cls` (representing the class itself) as the first parameter instead of `self`.

```
Example
class Dog:
    species = "Canis familiaris"
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed
    @classmethod
    def change_species(cls, new_species):
        print(f"This is dog species {cls.species}")
dog1 = Dog("Buddy", "Golden Retriever")
dog2 = Dog("Max", "Labrador")
print(dog1.species)  # Output: Canis lupus
print(dog2.species)  # Output: Canis lupus
```

---

2. Instance

An instance is an individual object created from a class. It has its own attributes (data) and methods (behavior).

1. Instance Arguments

Instance arguments are the values passed to the class constructor method `__init__()` when creating an object.

2. Instance Methods

Instance methods are functions defined inside a class that operate on instances(object) of that class. They have at least one parameter: `self`.

Example

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed
def bark(self):
        print(f"{self.name} says woof!")
dog1 = Dog("Max", "Labrador")
dog1.bark()  # Output: Max says woof!
```

---

3. Static

A **static method** is defined using the `@staticmethod` decorator. It behaves like a regular function but lives inside the class's namespace. Since it doesn't rely on instance or class variables, it doesn't require the `self` or `cls` parameter.

```
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b
result = MathUtils.add(5, 3)
print(result)  # Output: 8
```

In this example, `add` is a static method because it doesn't use any instance or class variables. You can call it using the class name directly, without creating an instance.

Characteristics (Features) of Object-Oriented Programming (OOP:

1. Encapsulation

Encapsulation **is** refers to the **bundling (packaging) of data (variables)** and the **methods (functions)** that

operate on that data **into a single unit** (a class), and **restricting access** to some of the object's internal state.

- Protect object data from unauthorized access.
- Hide internal implementation details from the user.

Access Modifiers in Python

Python doesn't have traditional **access modifiers** like `private`, `protected`, or `public`, but it uses naming conventions:

Modifier	Syntax	Meaning
Public	`var`	Accessible from anywhere
Protected	`_var`	Internal use only (not enforced)
Private	`__var`	Name mangled to prevent direct access

Public (`var`)

✅ Syntax: `var`

- No underscores before the variable or method name.
- Accessible **from anywhere** — both inside and outside the class.
- Commonly used for data or methods intended to be used freely.

✅ Example (Variable):

```
class Dog:
    def __init__(self):
        self.name = "Buddy"  # public attribute
dog = Dog()
print(dog.name)  # Output: Buddy
```

✅ Example (Method):

```
class Dog:
    def bark(self):
        print("Woof!")
dog = Dog()
dog.bark()  # Output: Woof!
```

Protected (`_var`)

✅ Syntax: `_var`

- A **single underscore _** before the variable or method name.
- Signals that it's for **internal use only**.
- **Still accessible** from outside, but considered **a convention** to treat it as non-public.

✅ Example (Variable):

```
class Dog:
    def __init__(self):
        self._age = 5  # protected attribute
dog = Dog()
print(dog._age)  # Output: 5 (allowed, but discouraged)
```

✅ Example (Method):

```
class Dog:
    def _growl(self):
        print("Grr...")
dog = Dog()
dog._growl()  # Output: Grr... (discouraged, but works)
```

🔒 Private (`__var`)

✅ Syntax: `__var`

- A **double underscore __** before the variable or method name.
- Python internally renames the variable to `_ClassName__var`.

✅ Example (Variable):

```
class Dog:
    def __init__(self):
        self.__secret = "I love India"  # private attribute
dog = Dog()
# print(dog.__secret)      # ❌ AttributeError
print(dog._Dog__secret)    # ✅ Output: I love India
```

✅ Example (Method):

```
class Dog:
    def __init__(self):
        pass
    def __whisper(self):
        print("Shhh... it's a secret")
dog = Dog()
# dog.__whisper()          # ❌ AttributeError
dog._Dog__whisper()        # ✅ Output: Shhh... it's a secret
```

2. Inheritance

Inheritance is the process(technique) by which **one class (child/derived class)** can **acquire properties and behaviors (methods/attributes)** from another **class (parent/base class).**It allows for **code reusability**, **hierarchical relationships**, and **extensibility** of classes.

Why Use Inheritance?

- **Reuse code** in multiple classes.
- Create a **hierarchy** of classes.
- Add or **override behavior** in child classes.
- Makes your code more **modular and manageable**.

1. Single Inheritance

```
class Animal:
    def speak(self):
        print("Animal speaks")
class Dog(Animal): #connect Animal class
    def bark(self):
        print("Dog barks")
d = Dog()
d.speak()  # Inherited from Animal
d.bark()   # Defined in Dog
```

2. Multilevel Inheritance

```
class Animal:
    def eat(self):
        print("Animal eats")
class Mammal(Animal):
    def walk(self):
        print("Mammal walks")
class Dog(Mammal):
    def bark(self):
        print("Dog barks")
d = Dog()
d.eat()    # From Animal
d.walk()   # From Mammal
d.bark()   # From Dog
```

3. Multiple Inheritance

```
class Father:
    def skills(self):
        print("Knows coding…")
class Mother:
    def skills(self):
        print("Knows cooking")
class Child(Father, Mother):
    def play(self):
        print("Loves to play")
```

```
c = Child()
c.play()
c.skills()  # ⚠️ Resolves to Father's method due to Method Resolution Order (MRO)
```

Method Resolution Order (MRO):
MRO (Method Resolution Order) **is the order in which Python looks for a method or attribute in a hierarchy of classes during method execution.**

Why MRO Matters

```
class A:
    def show(self):
        print("A")
class B(A):
    def show(self):
        print("B")
class C(A):
    def show(self):
        print("C")
class D(B, C):
    pass
d = D()
d.show()
```

Output:

```
B
```

How to View MRO

```
print(D.__mro__)
(<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>)
```

4. Hierarchical Inheritance

```
class Parent:
    def speak(self):
        print("Parent speaks")

class Son(Parent):
    def sing(self):
        print("Son sings")

class Daughter(Parent):
    def dance(self):
        print("Daughter dances")
s = Son()
d = Daughter()
s.speak()
d.speak()
```

`super()`?

`super()` is a built-in function that gives you access to methods in a **parent or sibling class**—as determined by the **Method Resolution Order (MRO)**.

Why Use `super()`?

- Avoids hardcoding parent class names
- Works cleanly with multiple inheritance
- Ensures proper MRO behavior
- Easier to maintain and refactor

Basic Syntax

```
super().method_name(args)
```

Example: Single Inheritance

```
class Parent:
    def show(self):
        print("Parent class")
class Child(Parent):
    def show(self):
        print("Child class")
        super().show()
obj = Child()
obj.show()
```

Output:

```
Child class
Parent class
```

Multiple Inheritance with `super()`

```
class A:
    def show(self):
        print("A")
        super().show()
class B(A):
    def show(self):
        print("B")
        super().show()
class C(A):
    def show(self):
        print("C")
        super().show()
class D(B, C):
    def show(self):
        print("D")
        super().show()
d = D()
d.show()
```

Output:

D
B
C
A

---

3. Polymorphism

Polymorphism means **"many forms"**. In OOP, it allows the **same method or function name** to work in **different ways** depending on the object calling it.
In simple terms, **you can use one name to do different things** depending on the object type.

Types of Polymorphism in Python

1. Compile-Time Polymorphism *(Static Polymorphism)*

Compile-time polymorphism refers to the ability of a function or method to behave differently based on the number or type of its arguments. It is resolved during the compilation phase.
⚠️ Python does **not support** traditional compile-time polymorphism like some statically typed languages (e.g., Java or C++), but similar behavior can be **simulated** using techniques like default arguments.

Example – Method Overloading (Simulated):

```
class Calculator:
    def add(self, a=0, b=0, c=0):
        return a + b + c
calc = Calculator()
print(calc.add(1, 2))       # Output: 3
print(calc.add(1, 2, 3))    # Output: 6
```

2. Run-Time Polymorphism *(Dynamic Polymorphism)*

Run-time polymorphism allows methods to be overridden in derived classes, and the method that gets executed is determined at **runtime** based on the object's actual type.

a. Method Overriding

A subclass provides a specific implementation of a method that is already defined in its parent class.

Example 1:

```
class Animal:
    def speak(self):
        print("Animal speaks")
class Dog(Animal):
    def speak(self):
        print("Dog barks")
class Cat(Animal):
```

```
    def speak(self):
        print("Cat meows")
def make_sound(animal):
    animal.speak()
make_sound(Dog())  # Output: Dog barks
make_sound(Cat())  # Output: Cat meows
```

Example 2:

```
class BankAccount:
    def get_interest_rate(self):
        return 5  # 5% default
class SavingsAccount(BankAccount):
    def get_interest_rate(self):
        return 7  # 7% for savings
class CurrentAccount(BankAccount):
    def get_interest_rate(self):
        return 3  # 3% for current
# Usage
print(SavingsAccount().get_interest_rate())  # 7
print(CurrentAccount().get_interest_rate())  # 3
```

b. Duck Typing

Duck typing is when you don't check an object's type — you just use it if it has the method you need.If something *acts* like what you need, you don't worry about what it *is*.

Example 1:

```
class Robot:
    def speak(self):
        print("Robot speaking")
class Human:
    def speak(self):
        print("Human speaking")
def greet(entity):
    entity.speak()  # No type check needed
greet(Robot())  # ✅ works
greet(Human())  # ✅ also works
```

We didn't check if `entity` is a `Robot` or `Human`. As long as it has a `speak()` method, it works. That's duck typing.

3. Operator Overloading

Operator Overloading **is a type of** polymorphism **where the** same operator **(like +, -, *, etc.) behaves** differently **depending on the** operands **(the objects it's used on).**
It allows you to define custom behavior for operators when they're used with user-defined classes**.**

Common Operators You Can Overload in Python

Operator	Special Method	Description
+	`__add__(self, other)`	Addition
-	`__sub__(self, other)`	Subtraction
*	`__mul__(self, other)`	Multiplication
/	`__truediv__(self, other)`	Division
==	`__eq__(self, other)`	Equality check
<	`__lt__(self, other)`	Less than comparison
>	`__gt__(self, other)`	Greater than comparison

---

4. Abstraction

Abstraction is focuses on **hiding complex internal details** and **showing only essential information** to the user.

Think of it like driving a car:

- You **use the steering wheel, pedals, and gear** (interface),
- But you don't know (or need to know) **how the engine or brakes are working internally**.
- **What** an object does, rather than **how** it does it.
- Helps in **reducing complexity** and **increasing code reusability**.

`abc` Module: `abc` stands for **Abstract Base Classes**. It's a built-in Python module that provides infrastructure for defining **abstract base classes**.
`ABC` : `ABC` is a **base class** that your class must inherit from to be considered an **abstract base class**.

```
from abc import ABC, abstractmethod
class Animal(ABC):
```

This means `Animal` is now treated as an **abstract class**, and you can't create an instance of it directly.

✅ Example Using `abc` Module

```
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass
```

```
    def sleep(self):
        print("Sleeping...")
# Subclass implementing abstract method
class Dog(Animal):
    def make_sound(self):
        print("Woof!")
class Cat(Animal):
    def make_sound(self):
        print("Meow!")
# Cannot instantiate Animal directly
# animal = Animal()  # This will raise an error
dog = Dog()
dog.make_sound()  # Output: Woof!
dog.sleep()       # Output: Sleeping...
cat = Cat()
cat.make_sound()  # Output: Meow!
```

Why Use Abstraction?

- Enforces a standard structure across subclasses.
- Promotes **code reusability** and **scalability**.
- Helps with **security** by exposing only relevant parts of the code.